

Universidad Nacional de Trujillo

Facultad de Ciencias Físicas y Matemáticas

Ingeniería Informática



Base de Datos Avanzada

Temas.





Universidad Nacional de Trujillo



Introducción

Introducción.

El análisis de grandes volúmenes de datos en un tiempo razonable requiere abundantes recursos informáticos, incluyendo unidades centrales de procesamiento, espacio en disco para el almacenamiento de datos, alojamientos en nube y ancho de banda de red. La tecnología de escalado horizontal inicio actividades de investigación y desarrollo (I+D) para abordar el análisis de grandes volúmenes de datos mediante la adición de servidores blade/rack para aumentar la velocidad de procesamiento y expandir la capacidad de datos.



Universidad Nacional de Trujillo



Desarrollo

1. Servicios avanzados mediante el procesamiento de datos a gran escala.

La escalabilidad horizontal ha reducido el costo de construir sistemas para procesar grandes volúmenes de datos.



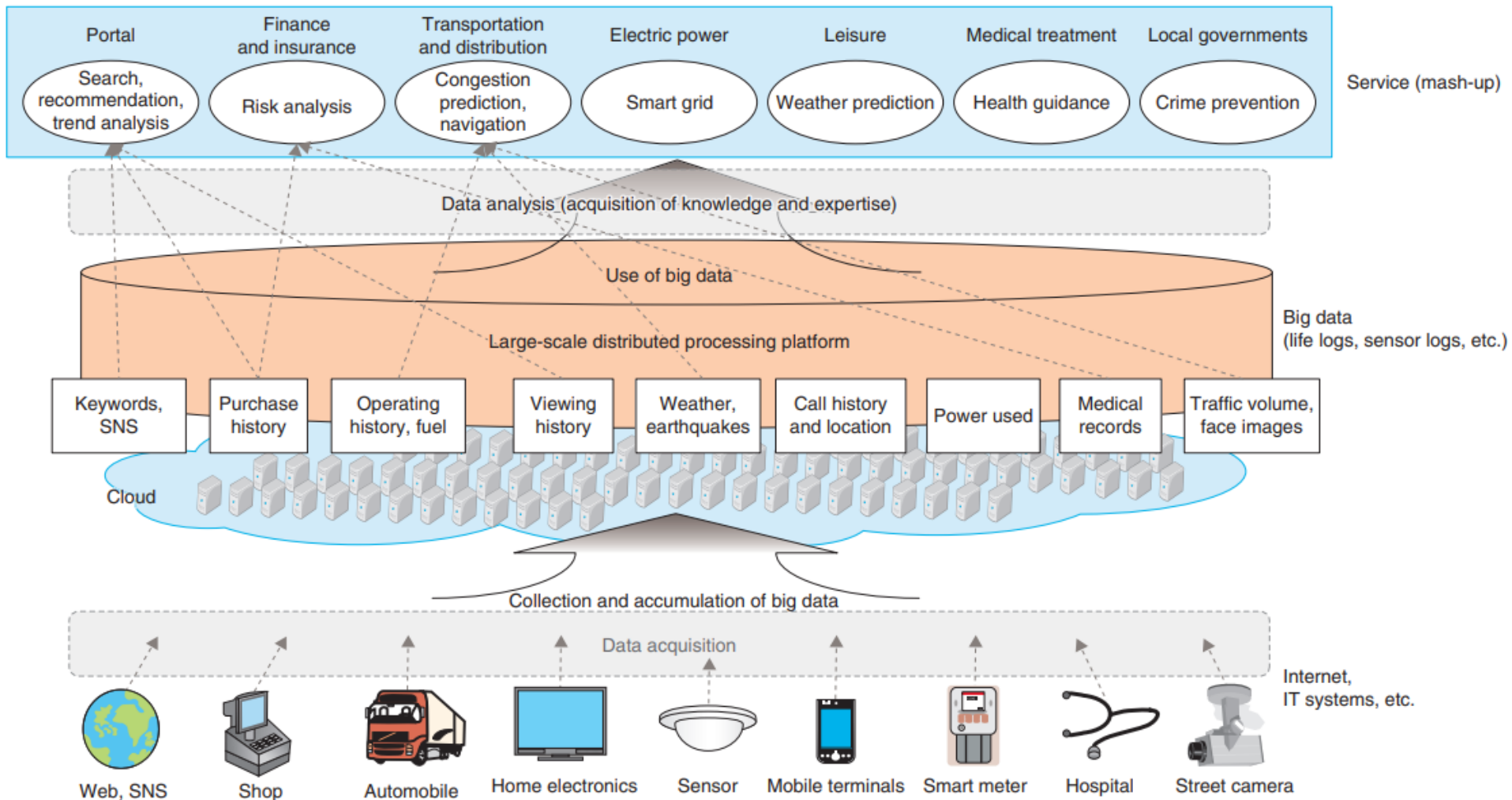
1. Servicios avanzados mediante el procesamiento de datos a gran escala.

Mash-up: Un nuevo servicio creado mediante la combinación de múltiples interfaces de programación de aplicaciones.

Una plataforma de procesamiento de datos distribuidos a gran escala recopila y almacena los macrodatos generados por sistemas informáticos o internet.

Al analizar estos grandes volúmenes de datos, se pueden adquirir nuevos conocimientos y experiencia, así como crear nuevos servicios combinados para generar conocimiento respecto a servicios avanzados para los clientes.

1. Servicios avanzados mediante el procesamiento de datos a gran escala.



SNS: social networking service

2. Desarrollo de la Optimización de Control Basado en Costos (CBoC)



El enfoque convencional para la gestión de diversos tipos de datos consiste en utilizar un sistema de gestión de bases de datos relacionales (RDBMS), pero para el Big data, la tecnología NoSQL, resulta una alternativa rentable para implementar la escalabilidad horizontal, al flexibilizar la garantía de consistencia de datos, es idóneo para el procesamiento de Big data menos estructurado, como los datos en lenguaje natural.

2. Desarrollo de la Optimización de Control Basado en Costos (CBoC)



3. Mejorar la tolerancia a fallos en plataformas de procesamiento de datos distribuidos a gran escala



CBoC comprende tres subsistemas de procesamiento distribuido:

1. Un sistema de archivos distribuido para almacenar grandes cantidades de datos en muchos servidores blade/rack de forma distribuida
2. Un subsistema de tablas distribuidas para gestionar grandes cantidades de datos como datos estructurados
3. Un subsistema de bloqueo distribuido, que proporciona una función básica que permite un alto grado de resolución en estos sistemas distribuidos

3. Mejorar la tolerancia a fallos en plataformas de procesamiento de datos distribuidos a gran escala



3. Mejorar la tolerancia a fallos en plataformas de procesamiento de datos distribuidos a gran escala



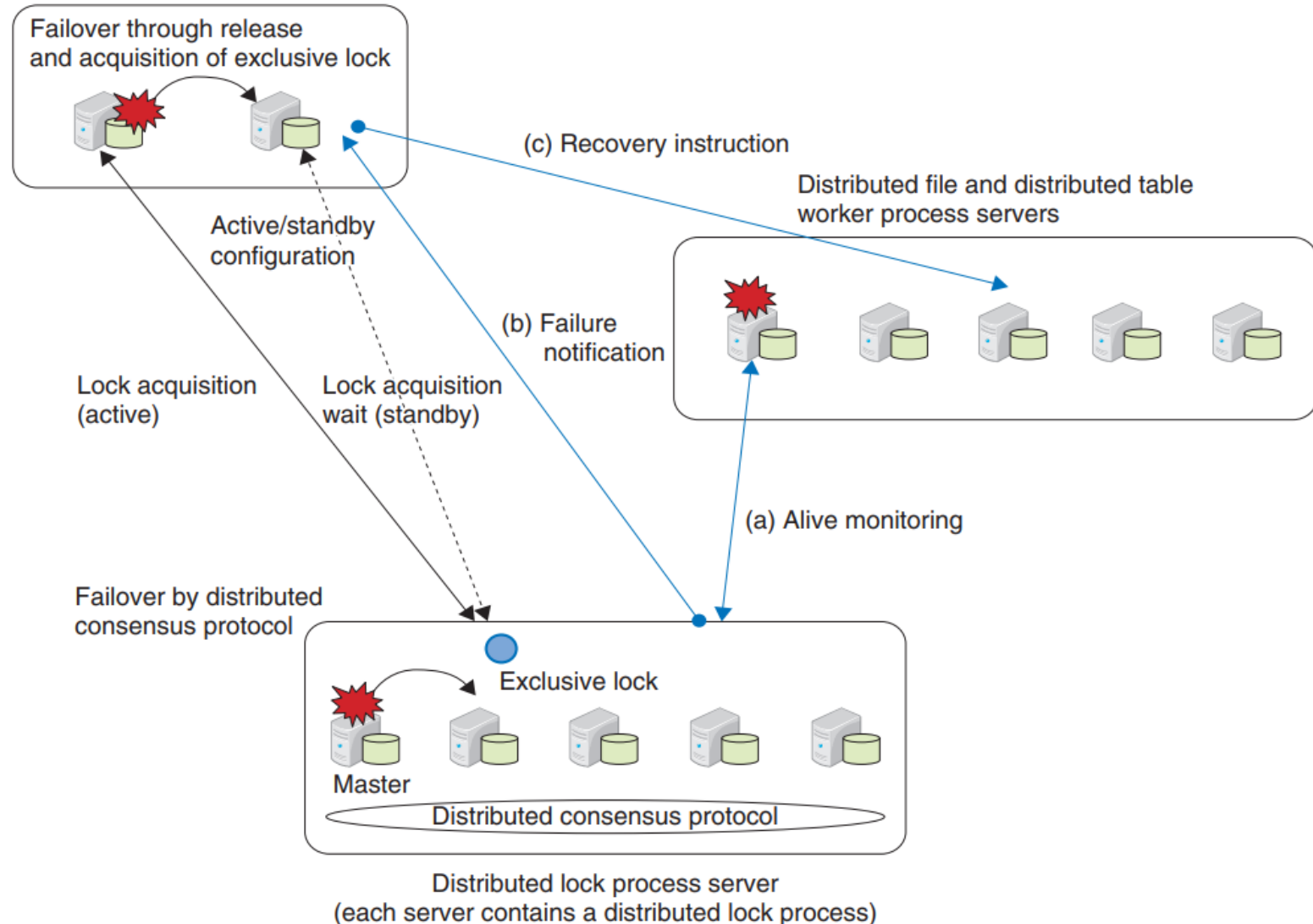
CBoC, mejoró la tolerancia a fallos, esto fue importante para gestionar la escalabilidad horizontal. La función de bloqueo distribuido proporciona la base para la tolerancia a fallos en los subsistemas de archivos y tablas distribuidas

En sistemas que utilizan muchos servidores blade/rack, la probabilidad de un fallo general del sistema es alta, incluso si la tasa de fallos de los servidores individuales es baja.

3. Mejorar la tolerancia a fallos en plataformas de procesamiento de datos distribuidos a gran escala

Figura: 3

- Distributed file, distributed table master process server



3.1. Tolerancia a fallos de los bloqueos distribuidos

La tolerancia a fallos de los bloqueos distribuidos es la capacidad de un sistema para mantener la correcta coordinación y exclusión mutua de los recursos compartidos aun cuando ocurran fallos de nodos, procesos, redes o comunicaciones. Su objetivo principal es evitar situaciones en las que un recurso quede bloqueado indefinidamente o sea utilizado simultáneamente por múltiples procesos debido a errores del sistema.

Los mecanismos de bloqueo distribuido suelen emplear técnicas como ***tiempos de expiración*** (timeouts), ***replicación de información de bloqueo***, ***detección de fallos mediante heartbeats***, ***elección de líderes*** y ***protocolos de consenso***. Estas estrategias permiten liberar automáticamente bloqueos huérfanos, recuperar el control tras una caída y garantizar la consistencia del sistema.

3.1. Tolerancia a fallos de los bloqueos distribuidos

TOLERANCIA A FALLOS EN SISTEMAS DISTRIBUIDOS



3.2. Tolerancia a fallos de archivos y tablas distribuidas

- ✓ ***Los archivos y tablas distribuidos***, constan de numerosos procesos de trabajo que procesan las solicitudes de las aplicaciones y dos procesos maestros que controlan dichos procesos.
- ✓ ***El proceso maestro activo y el maestro en espera***, se distinguen mediante la función de bloqueo exclusivo del bloqueo distribuido. El maestro que logra adquirir un bloqueo exclusivo se convierte en el maestro activo, y el que no lo consigue se convierte en el maestro en espera.
- ✓ ***El bloqueo distribuido***, monitoriza el estado de cada proceso maestro; si el maestro activo falla, se libera el bloqueo exclusivo.
- ✓ ***El maestro en espera***, puede entonces adquirir el bloqueo exclusivo y convertirse en el maestro activo (conmutación por error).

3.2. Tolerancia a fallos de archivos y tablas distribuidas

El bloqueo distribuido también supervisa el estado de los procesos de trabajo.

- a) ***Si un proceso de trabajo falla***, se notifica al maestro activo
- b) ***El maestro activo***, da instrucciones a otro proceso de trabajo en ejecución para recuperar los datos que gestionaba el proceso fallido
- c) ***Este último***, retoma el procesamiento que estaba realizando el proceso fallido. Para evitar la pérdida de datos debido a fallos en los archivos distribuidos, los datos se vuelven redundantes y son gestionados por varios procesos de trabajo.

3.3. Tolerancia a fallos considerando la red

- ✓ La configuración de red de los servidores también influye notablemente en la tolerancia a fallos. CBoC utiliza una configuración de red en árbol, donde varios conmutadores de borde están subordinados a un conmutador central, y cada conmutador de borde se conecta a varios servidores.
- ✓ Los procesos maestros del sistema de archivos y tablas distribuidos se ubican bajo diferentes conmutadores de borde, y los procesos de trabajo gestionan los datos redundantes de los archivos distribuidos, ubicados bajo distintos conmutadores de borde.
- ✓ De esta forma, CBoC implementa la tolerancia a fallos de manera que todo el sistema no se caiga cuando falla una sola unidad de hardware (como un servidor o un conmutador), incluso si un fallo en un conmutador de borde provoca que varios servidores se desconecten del sistema al mismo tiempo.

3.3. Tolerancia a fallos considerando la red



4. Pruebas de sistemas en sistemas distribuidos a gran escala



- ✓ Los sistemas distribuidos a gran escala diseñados para el análisis de macrodatos pueden constar de decenas a cientos de servidores, o incluso miles en algunos casos, dependiendo de la magnitud de los datos a procesar y la naturaleza del procesamiento.
- ✓ El número de variaciones de transiciones de estado en estos sistemas distribuidos es enorme, por lo que las pruebas del sistema son, en consecuencia, aún más importantes.

4. Pruebas de sistemas en sistemas distribuidos a gran escala



4.1. Automatización de la construcción del entorno de pruebas

- ✓ La dificultad de construir entornos de prueba se puede apreciar simplemente considerando el trabajo que implica instalar software en cientos de servidores.
- ✓ Las herramientas de automatización de la gestión de sistemas pueden utilizarse eficazmente en la construcción de dichos entornos.
- ✓ La construcción y el mantenimiento de entornos pueden optimizarse mediante el uso de herramientas de automatización de la gestión de sistemas como Puppet (herramienta de administración de configuración de código abierto)

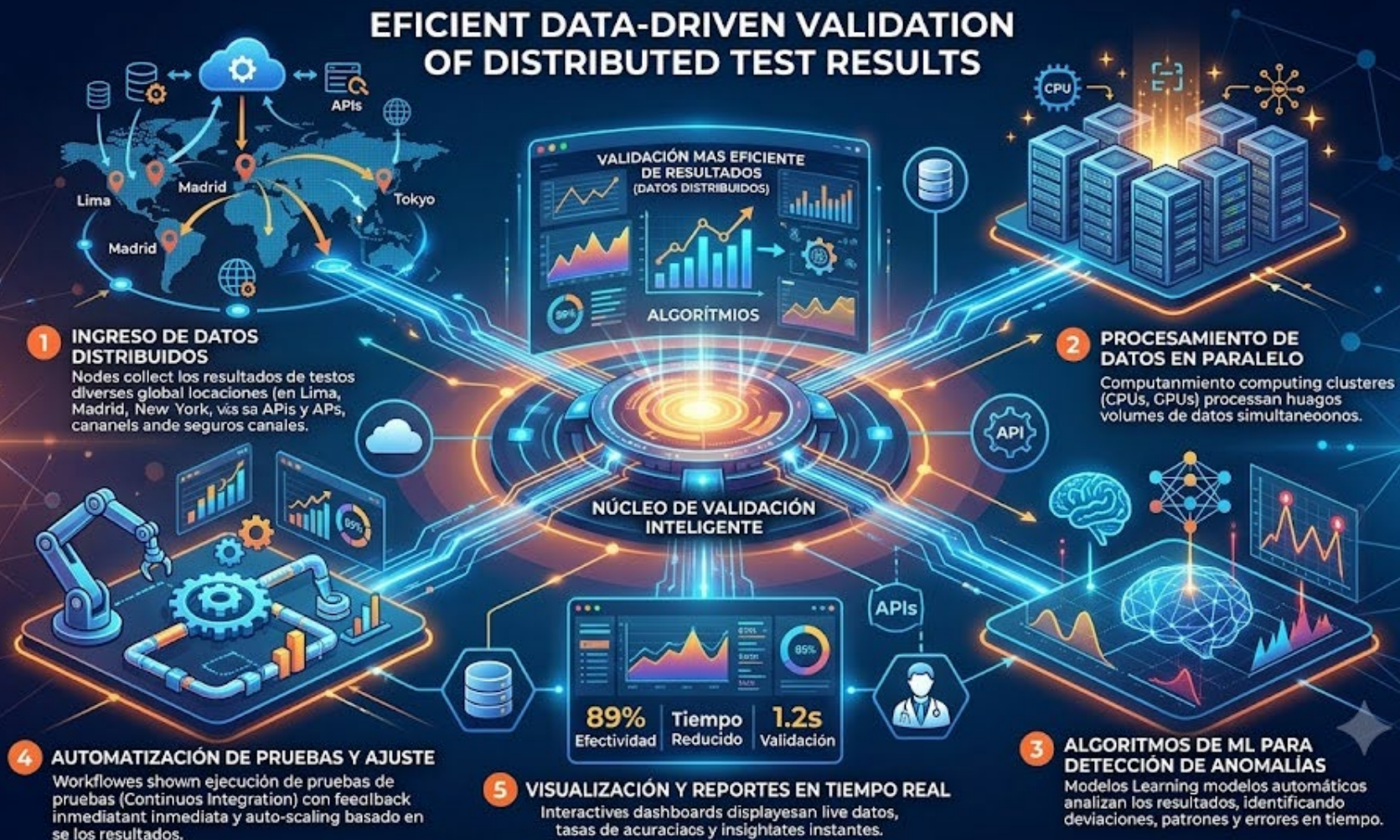
4.2. Entrada más rápida de macrodatos

- ✓ Para verificar el funcionamiento estable del sistema, desarrollamos una herramienta de entrada de datos que permite crear diversos estados de almacenamiento en poco tiempo. Esta función es esencial para realizar pruebas cuando se ha introducido una gran cantidad de datos.
- ✓ La herramienta incorpora funciones para reiniciar el proceso desde un estado intermedio y para añadir automáticamente los datos introducidos previamente, entre otras. Estas funciones reducen considerablemente el tiempo de preparación necesario para las pruebas.

4.3. Validación más eficiente de los resultados de las pruebas



- ✓ Los elementos involucrados en la validación de los resultados de las pruebas incluyen los resultados de entrada y salida vistos por la aplicación, el estado de entrada de datos y si se produjo o no un error.
- ✓ Para entornos de pruebas a gran escala en particular, es importante la observación y visualización en tiempo real de la ocurrencia de errores y los estados de los procesos del servidor, lo cual es posible gracias al monitoreo de la operación del sistema, el registro de errores y otros registros similares.





Actividad de Aprendizaje

Fase 1: Autoaprendizaje (Investigación individual)

1. ¿Cuál es la relación entre la computación en la nube y los modelos de procesamiento distribuido como Hadoop o Spark en el contexto de Big Data?
2. ¿Qué diferencias fundamentales existen entre los modelos ACID y BASE, y cómo impactan en el diseño de sistemas modernos?
3. Investiga cómo funcionan los algoritmos de consenso distribuido (por ejemplo, Paxos o Raft) y explica su importancia en la tolerancia a fallos.
4. ¿Qué papel cumplen los sistemas de archivos distribuidos (como HDFS) en la gestión de grandes volúmenes de datos y cómo optimizan el acceso?
5. Analiza casos reales de empresas (como Google, Amazon o Netflix) que utilicen procesamiento de datos a gran escala y explica qué beneficios obtienen.

Fase 2: Discusión (Debate crítico grupal)

1. ¿Es más importante la consistencia o la disponibilidad en sistemas de Big Data? Justifiquen su postura considerando distintos escenarios.
2. ¿La adopción de tecnologías NoSQL representa una evolución o una limitación frente a los sistemas relacionales tradicionales?
3. ¿El modelo de escalabilidad horizontal es siempre la mejor solución frente al escalado vertical? Argumenten con ejemplos.
4. ¿Hasta qué punto la tolerancia a fallos justifica el incremento en la complejidad de los sistemas distribuidos?
5. ¿El uso de herramientas de automatización (como Puppet) reemplaza completamente la intervención humana en la gestión de sistemas?

Fase 3: Reflexión (Síntesis grupal)

1. ¿Qué postura final adoptó el grupo respecto al equilibrio entre consistencia, disponibilidad y escalabilidad?
2. ¿Qué argumentos fueron más sólidos durante el debate y por qué?
3. ¿Cómo cambió su comprensión del Big Data y los sistemas distribuidos después de la discusión?
4. ¿Qué aspectos del tema consideran más relevantes para su futura vida profesional?
5. ¿Qué dudas o temas quedaron abiertos y requieren mayor profundización?



Universidad Nacional de Trujillo



Cierre

Gracias



Universidad Nacional de Trujillo